



Linear Algebraic Depth-First Search

Daniele G. Spampinato
Department of Electrical and
Computer Engineering
Carnegie Mellon University
Pittsburgh, PA, USA
spampinato@cmu.edu

Upasana Sridhar
Department of Electrical and
Computer Engineering
Carnegie Mellon University
Pittsburgh, PA, USA
upasanas@cmu.edu

Tze Meng Low
Department of Electrical and
Computer Engineering
Carnegie Mellon University
Pittsburgh, PA, USA
lowt@cmu.edu

Abstract

There is a recent push by a segment of the graph community to implement graph algorithms in the language of linear algebra. However, graph algorithms that depend on depth-first search (DFS) techniques are often highlighted as limitations of the linear algebraic approach as linear algebraic formulation of DFS algorithms are few, if any. This paper provides a linear algebraic approach for developing DFS graph algorithms and demonstrates its use for defining three classical DFS-based computations: Binary tree traversal, topological sort, and biconnected components.

CCS Concepts • Mathematics of computing → Graph algorithms; • Theory of computation → Algorithm design techniques; *Sorting and searching*; Hoare logic; • Computing methodologies → Linear algebra algorithms.

Keywords graph algorithms, linear algebra, adjacency matrix, permutations, depth-first search, pre-order traversal, post-order traversal, topological sort, biconnected components

ACM Reference Format:

Daniele G. Spampinato, Upasana Sridhar, and Tze Meng Low. 2019. Linear Algebraic Depth-First Search. In *Proceedings of the 6th ACM SIGPLAN International Workshop on Libraries, Languages and Compilers for Array Programming (ARRAY '19)*, June 22, 2019, Phoenix, AZ, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3315454.3329962>

1 Introduction

The GraphBLAS [5] is a recent community-driven effort to provide a standard interface for the implementation of graph

algorithms in the language of linear algebra. A key objective of this effort is to provide a standardized interface that allows for a separation of concerns for both the algorithm writer and the graph library developer. Algorithm writers can develop their graph algorithms using this standardized interface based on the language of linear algebra while library developers have a standardized interface for which to tune their library implementations [2, 10, 11]. In addition, using the language of linear algebra as the interface also allows library developers to leverage decades of expertise from optimizing sparse linear algebra libraries to develop high performance graph libraries.

While a large number of graph algorithms [6, 8, 9] have been expressed and implemented in the language of linear algebra, many of the algorithms are based on the breadth-first-search traversal pattern. In contrast, to the best of the authors knowledge, there are no discussions on how to formulate depth-first-search (DFS) graph algorithms in the language of linear algebra. As DFS is a core functionality for many algorithms (e.g., tree and graph traversal, and the identification of structures such as biconnected components [1, 4, 7]), the lack of a linear algebraic approach for computing DFS could potentially result in a large class of algorithms being excluded from GraphBLAS. As such, it is vital for the graph community that there exist a systematic methodology for formulating DFS graph algorithms in the language of linear algebra.

In this paper, we make the observation that a DFS algorithm imposes an order in the way the vertices of a graph are visited. Specifically, at the start of a DFS algorithm, vertices are *unordered*. During the course of its execution, vertices can be described either as *ordered*, *partially ordered*, and *unordered*. Vertices that are *ordered* will not be ordered again. A vertex is *partially ordered* when an initial visit order is assigned to the vertex, but that order in which the vertex is visited may be changed later in the DFS traversal. Vertices that are *unordered* have no visit order assigned. The DFS algorithm terminates when all vertices have been *ordered* as desired. This implies that a DFS algorithm can be implemented by tracking the status of the vertices and the order in which they are visited.

The above observation suggests that an array-based approach towards implementing DFS algorithms can take the following approach:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. ARRAY '19, June 22, 2019, Phoenix, AZ, USA

© 2019 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-6717-2/19/06...\$15.00

<https://doi.org/10.1145/3315454.3329962>

- Consider an array that has been divided into three (possibly empty) sections. Let the top-most section hold *ordered* vertices, *partially ordered* vertices are located in the middle section, while the *unordered* vertices are in the bottom section.
- During the execution of DFS algorithms, two possible actions may occur:
 1. One or more *partially ordered* vertices are moved into the *ordered* section; and/or
 2. One or more *unordered* vertices are assigned an initial partial visit order, and thus moved into *partially ordered* section.
- The algorithm terminates when all vertices are in the *ordered* section of the array.

In the rest of this paper, we will formally develop this approach within the confines of the language of linear algebra. We demonstrate the use of this approach by providing a linear algebraic formulation of three classical DFS-based algorithms of increasing complexity: Binary tree traversal, topological sort, and finding biconnected components.

2 Graphs in Linear Algebra

A graph $\mathcal{G} = (V, E)$ can be represented in the language of linear algebra by its adjacency matrix, $A_{\mathcal{G}}$. While a graph can also be represented with its incidence matrix, we limit our discussion in this paper to only the adjacency matrix representation.

Adjacency matrix. The adjacency matrix of a graph $\mathcal{G} = (V, E)$ is a $|V| \times |V|$ matrix, where the presence of an element at the (i, j) position of $A_{\mathcal{G}}$ represents the edge, (v_i, v_j) , from vertex v_i to vertex v_j . For simplicity, we will assume that $A_{\mathcal{G}}$ is a binary matrix, where the value of 1 represents the presence of an edge, while a 0 means that the two vertices are not connected. Using this representation, edges in-coming and out-going from a vertex v_i are located in the i^{th} column and i^{th} row of $A_{\mathcal{G}}$ respectively.

Identifying vertices and their edges. Notice that in representing a graph with its adjacency matrix, vertices are assigned a unique number from the set $\{1, \dots, |V|\}$. It is thus convenient for us to associate each vertex v_i with the column vector e_i , where e_i is a binary vector of length $|V|$, and the i^{th} element of e_i is a one, while the remaining elements are zeros. e_i is also the i^{th} basis vector from the standard basis for $\mathbb{N}^N$.

We can identify in-coming and out-going edges in the vertex v_i by performing a matrix-vector multiplication of e_i with the adjacency matrix, $A_{\mathcal{G}}$. In-coming edges of v_i are found with the operation

$$y_{in} = A_{\mathcal{G}} e_i,$$

which picks out the i^{th} column of $A_{\mathcal{G}}$. Similarly, out-going edges of v_i are identified with

$$y_{out} = A_{\mathcal{G}}^T e_i, \quad (1)$$

which picks out the i^{th} row of $A_{\mathcal{G}}$.

The predecessor and successor vertices of vertex v_i are also identified with the above operations. Recall that each vertex is associated with a basis vector, $e_j, j \in \{1, \dots, |V|\}$. This means that the vectors y_{in} and y_{out} can be viewed as a sum of basis vectors, where each of the basis vector that contributes to y_{in} and y_{out} are the predecessors and successors of v_i . The number of predecessor and successors can be found by performing an inner (or dot) product, i.e.

$$y_{in}^T y_{in} \text{ and } y_{out}^T y_{out}$$

respectively.

Permutation matrices. A permutation matrix is an $N \times N$ binary matrix where every row and column has $N - 1$ zeros, and a single one. Since each column of the matrix has exactly a single one, we can represent a permutation matrix P as N basis vector as such:

$$P = (e_{\pi(1)} \mid \dots \mid e_{\pi(N)}),$$

where $\pi(i) \in \{1, \dots, N\}, \forall i, j : i \neq j \implies \pi(i) \neq \pi(j)$. For instance, the permutation matrix

$$P = (e_3 \mid e_1 \mid e_2) = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}. \quad (2)$$

The permutation matrix where the basis vectors are in their natural order, i.e.

$$P = (e_1 \mid \dots \mid e_N) = I_N$$

is known as the identity matrix, and is assigned the symbol I_N . Here, the subscript N represents the size of the permutation matrix.

A permutation matrix P is used to reorder the elements of a vector u . Specifically, by performing a matrix vector multiplication, i.e.,

$$v = P^T u,$$

the elements of the input u are reordered such that in the resulting vector v , the elements are ordered in the same manner as the basis vectors that form P . Using the permutation matrix P in (2), the elements in vector $u = (1, 2, 3)^T$ are reordered as $v = (3, 1, 2)^T$.

Reordering adjacency matrices. A permutation matrix can similarly be used to reorder rows and columns of a matrix A . The rows of a matrix A are reordered by multiplying A by P^T , i.e., $P^T A$. In a similar manner, columns can be reordered by multiplying A from the right with P .

Within the context of linear algebraic graph algorithms, both rows and columns of the adjacency matrix must be reordered in the same manner to preserve the property that the $(i, j)^{th}$ element of $A_{\mathcal{G}}$ represents an edge from v_i and

v_j . This can be achieved by pre- and post-multiplying the adjacency matrix A_G , with the same permutation matrix P in the following manner

$$B = P^T A_G P.$$

Composing permutations. Permutation matrices are composable. This means that applying two permutation matrices P_1 and P_2 in sequence is the same as applying the permutation matrix P , where

$$P = P_1 P_2.$$

3 Permutation-based DFS

In this section, we will demonstrate how the linear algebra operations discussed in the previous section, can be used to described DFS.

3.1 Status of Vertices

Through the course of an entire DFS algorithm, vertices are classified into three different statuses:

- *Unordered.* Vertices are unordered when the order in which they are visited have not been determined. All vertices are assigned the *unordered* status at the start of the algorithm.
- *Partially Ordered.* Vertices are deemed to be *partially ordered* when an initial visit order for the vertex has been assigned. However, this initial order may be updated depending on the computation that must be performed. A *partially ordered* vertex may be visited multiple times, where a new visit order may be assigned to the vertex each time.
- *Ordered.* A vertex is said to be *ordered* when the order in which it is processed is fixed. Typically, this status is assigned to a vertex when the vertex will not be visited again in the future. The DFS algorithm terminates when all vertices are *ordered*.

We can keep track of the statuses of all vertices with an array that is represented by the column vector $s \in \mathbb{N}^N$, where $N = |V|$. Each element of s is the unique label of that has been assigned to a vertex. Assume that s has been partitioned into three (possibly empty) sub-vectors as follows:

$$s \rightarrow \left(\begin{array}{c} s_S \\ \hline s_P \\ \hline s_U \end{array} \right). \quad (3)$$

Here, the arrow indicates that s is viewed as three sub-vectors s_S , s_P , and s_U . The thick lines represents the boundaries between the different sub-vectors of s . The sub-vectors, s_S , s_P , and s_U correspond to the ordered set of vertices with the status *ordered*, *partially ordered*, and *unordered* respectively. In addition, we also define τ and β as follows

$$\tau = |s_S| + 1 \quad (4)$$

$$\beta = |s_S| + |s_P| + 1 \quad (5)$$

In addition, as vertices are visited and processed, the size of s_S will grow while s_U will shrink. When all vertices are in s_S , and s_U is empty, i.e. $\tau = \beta = |N| + 1$, the DFS algorithm terminates.

As an example, the following vector s :

$$s \rightarrow \left(\begin{array}{c} \sigma_1 \\ \sigma_2 \\ \hline \sigma_3 \\ \vdots \\ \sigma_N \end{array} \right),$$

contains two *ordered* vertices whose labels are σ_1 and σ_2 . There are no *partially ordered* elements (i.e., s_P is empty) and the remaining $N - 2$ elements are still *unordered*.

3.2 Tracking Vertex Visit Order

Recall that DFS algorithms impose a specific order in which vertices are visited. We preserve this order in the DFS algorithm by the order in which the elements are stored in the s_S and s_P sub-vectors. Vertices whose labels are closer to the top of the sub-vector s_S are deemed to be visited first in the DFS algorithms. Similarly, vertices whose labels are near the top of s_P have visit orders earlier than vertices whose labels are lower down in the s_P . However, recall that vertices in s_P are *partially ordered*. This means the visit order for vertices in s_P may change as the algorithm proceeds.

Moving vertices from s_U to s_P . Recall that vertices start off having the status *unordered*. In order to move vertices from s_U into s_P , a number of steps must occur:

1. *Identifying vertices to move.* Vertices that are moved are identified based on certain properties. We use v to indicate the number of vertices that are moved. In moving vertices from s_U to s_P , some vertices will still remain in s_U . These vertices need to be collected, and will form the new s_U . The splitting of s_U into two distinct sets of vertices can be achieved with the permutation matrix P_1 of size $(N - \tau + 1) \times (N - \tau + 1)$.
2. *Partial ordering of vertices.* An ordering of the newly identified vertices and the existing *partially ordered* vertices have to be performed. This new ordering of the vertices can be implemented as a permutation matrix P_2 . In addition, we know that P_2 is of the size $n \times n$ where $n = \beta - \tau + v$.
3. *s_S remains unchanged.* As there are no new vertices introduced into s_S , the order of elements in s_S can be preserved with the permutation matrix $I_{\tau-1}$.
4. *Moving the boundary.* Since v new elements are removed from s_U and added to s_P , the boundary between the two sub-vectors must be lowered by v positions. This also means that β is now increased by v , i.e. $\beta \leftarrow \beta + v$.

We can summarize all the required vertex reordering with a single permutation matrix P where

$$P = \begin{pmatrix} I_{\tau-1} & 0 \\ 0 & P_1 \end{pmatrix} \begin{pmatrix} I_{\tau-1} & 0 & 0 \\ 0 & P_2 & 0 \\ 0 & 0 & I_{N-\beta-\nu+1} \end{pmatrix},$$

with P_1 , P_2 and the identity matrices are defined previously.

Moving vertices from s_P to s_S . A permutation matrix for moving vertices from *partially ordered* to *ordered* status can be similarly constructed. In this case, because these ν new vertices are processed after vertices in s_S , they only need to be ordered within themselves and placed immediately after vertices in s_S . This can be represented with the permutation matrix

$$P = \begin{pmatrix} I_{\tau-1} & 0 & 0 \\ 0 & P_1 & 0 \\ 0 & 0 & I_{N-\beta+1} \end{pmatrix},$$

where P_1 is the permutation matrix that identifies the ν vertices and compacts the remaining vertices in s_P . This means that the size of P_1 is $\beta \times \beta$. Finally, because new vertices are placed into s_S , τ has to be updated such that $\tau \leftarrow \tau + \nu$.

3.3 Stack Operations with Permutations

DFS algorithms on graphs are typically implemented with a stack data structure. We show how stack operations be can described in the language of linear algebra with permutation matrices. The core idea is to use the sub-vector s_P as the stack, and the element that is closest to the s_S region is the top of the stack. Vertices that are popped from the stack will be stored in s_S , while unprocessed vertices form s_U .

Checking if stack is empty. The stack is empty if there are no elements in s_P , i.e. $|s_P| = 0$. This means that a stack is empty if $\beta - \tau = 0$.

Peeking at the top of the stack. Assuming the stack is not empty, the peek operation returns the top element of the stack, which we have defined to be the top-most element in s_P . This element can be retrieved with the operation

$$\sigma_\tau = s^T e_\tau,$$

which selects the τ^{th} element of s , which is also the top-most element of s_P .

Popping the stack. Popping the stack involves moving the top element in s_P into s_S . Since the top of the stack is defined to the element closest to s_S , moving the element from s_P to s_S can be performed with an increase in the value of τ , i.e. $\tau = \tau + 1$.

Pushing onto the stack. Pushing an element onto the top of the stack involves finding an element in s_U , and then moving that particular element to the top of s_P .

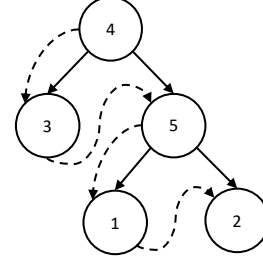


Figure 1. Example of binary tree with a pre-order traversal indicated with dashed arrows.

4 Linear Algebraic DFS on Binary Trees

In this section we consider the case where DFS is used to traverse binary trees. A binary tree $\mathcal{G} = (V, E)$ is characterized by having vertices with at most two children (i.e., the set of vertices in V directly connected to v by an edge in E) as shown in Fig. 1. DFS starts exploring a tree from an appointed vertex called the root and proceeds exploring every path originating from root as deep as possible, i.e., the sibling of a vertex v is visited only after all children of v have been visited. The depth-first traversal is normally guaranteed by the use of a stack to impose the order in which vertices are visited as shown in Algorithm 1. In particular, we first focus on the pre-order traversal of the tree. The input

Algorithm 1 Pseudocode for the iterative, pre-order DFS traversal of tree $\mathcal{G} = (V, E)$ with root $\rho \in V$.

```

function  $\ell$ : LIST = PREORDERTREE TRAVERSAL( $\mathcal{G}, \rho$ )
  Stack S
  S.push( $\rho$ )
  while not S.empty() do
     $v \leftarrow$  S.pop()
     $\ell.append(v)$ 
    for all  $e \in \mathcal{G}.E : e = (v, w)$  do
      S.push( $w$ )
    end for
  end while
end function

```

to the algorithm is the tree $\mathcal{G} = (V, E)$ to be traversed and the root vertex $\rho \in V$. The algorithm first pushes the root on the stack and then at each iteration it pops a vertex v from the top of the stack, adds it to the output list and pushes all its children on the stack. Fig. 1 shows a simple application of Algorithm 1 highlighting the traversal order using a dashed arrow. Next, we discuss our first example of linear algebraic formulation.

4.1 Pre-Order Tree Traversal

Algorithm 2 summarizes our permutation-based, linear algebraic pseudocode. We will explain the algorithm assuming

we want to explore the binary tree in Fig. 1 using a pre-order traversal from root $\rho = 4$. We start by representing the set of nodes in V using a vector $s \in \mathbb{N}^5$ and setting up the partially ordered markers at $[\tau, \beta] = [1, 1]$ as show below

$$s = \begin{pmatrix} \overline{1} \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}.$$

This is equivalent to saying that all elements are initially unordered, i.e., s_U in (3) equals s and $s_P = s_S = \emptyset$.

Step 1) Push root to s_P . The algorithm starts by placing the root vertex at the top of s_P by using a vector permutation and moving the bottom line as discussed in Sec. 3:

$$s \xrightarrow{s_1 = P_1^T s} \begin{pmatrix} \overline{4} \\ 1 \\ 2 \\ 3 \\ 5 \end{pmatrix} \xrightarrow{\beta = \beta + 1} \begin{pmatrix} \overline{4} \\ 1 \\ 2 \\ 3 \\ 5 \end{pmatrix},$$

where s_i and P_i are the vector and permutation matrices used at iteration i . We will address shortly how to form permutation matrices for pre-order as well as other forms of traversals. For now it suffices to assume that such permutation exists.

Step 2) Pop. Next, the algorithm enters its main loop and performs the first operation, i.e., visiting the top vertex by popping it. This means moving forward the top line, which in the first iteration is the root, and adding the vertex to s_P :

$$\begin{pmatrix} \overline{4} \\ 1 \\ 2 \\ 3 \\ 5 \end{pmatrix} \xrightarrow{\tau = \tau + 1} \begin{pmatrix} \overline{4} \\ 1 \\ 2 \\ 3 \\ 5 \end{pmatrix}$$

Step 3) Push. After popping the top, its children (3 and 5 in the example) are pushed into s_P . This operation is performed via permutation as in Step 1 above, i.e.,

$$\begin{pmatrix} \overline{4} \\ 1 \\ 2 \\ 3 \\ 5 \end{pmatrix} \xrightarrow{s_2 = P_2^T s_1; \beta = \beta + 2} \begin{pmatrix} \overline{4} \\ 3 \\ 5 \\ 1 \\ 2 \end{pmatrix}. \quad (6)$$

Repeat Step 2 and 3. Vertex 3 appears now at the top of s_P . Being a leaf, it does not have any children and the bottom line β does not move at the third iteration. The algorithm repeats the execution of Steps 2 and 3 and terminates when

all vertices appear at the top of vector s , i.e., $s_P = s_U = \emptyset$,

$$\begin{aligned} s_2 &\xrightarrow{\tau = \tau + 1} \begin{pmatrix} \overline{4} \\ 3 \\ 5 \\ 1 \\ 2 \end{pmatrix} \xrightarrow{\tau = \tau + 1} \begin{pmatrix} \overline{4} \\ 3 \\ 5 \\ 1 \\ 2 \end{pmatrix} \xrightarrow{s_3 = P_3^T s_2; \beta = \beta + 2} \begin{pmatrix} \overline{4} \\ 3 \\ 5 \\ 1 \\ 2 \end{pmatrix} \\ &\xrightarrow{\tau = \tau + 1} \begin{pmatrix} \overline{4} \\ 3 \\ 5 \\ 1 \\ 2 \end{pmatrix} \xrightarrow{\tau = \tau + 1} \begin{pmatrix} \overline{4} \\ 3 \\ 5 \\ 1 \\ 2 \end{pmatrix}. \end{aligned}$$

Next we refine the formulation of our algorithm basing it on the adjacency matrix of the tree.

Permuting the adjacency matrix. Algorithm 2 provides an implicit computation of the children of σ_τ . An explicit formulation can be obtained by reformulating our algorithm so that the input tree is represented using its adjacency matrix A_G . Notice that if the rows of A_G are arranged according to the order of s (i.e., the edges with source σ_i are located on the i th row of A_G) then we can compute the children of σ_i as described in (1), i.e., by first computing the vector of successor vertices $n = A_G^T e_i$. Further, if we rearrange the adjacency matrix A and s according to the same permutation, i.e., we update $s = P^T s$ and $A_G = P^T A_G P$. Therefore, using lines to split s into subregions induces a 2-D partitioning of A_G . For instance, consider the adjacency matrix of the binary tree in Fig. 1 provided below with rows labelled according to $s = (1\ 2\ 3\ 4\ 5)^T$,

$$A_G = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & 1 & \cdot & 1 \\ 1 & 1 & \cdot & \cdot & \cdot \end{pmatrix}. \quad (7)$$

Also, consider the permutation matrix P that takes s into the state in (6):

$$\begin{pmatrix} s_S \\ s_P \\ s_U \end{pmatrix} = \begin{matrix} \tau \\ \beta \end{matrix} \begin{pmatrix} \overline{4} \\ 3 \\ 5 \\ 1 \\ 2 \end{pmatrix} \quad (8)$$

$$\Downarrow$$

$$\begin{matrix} \tau & \beta \\ \begin{pmatrix} A_S & A_{SP} & A_{SU} \\ 0 & 0 & A_{PU} \\ 0 & 0 & A_U \end{pmatrix} \end{matrix} = \begin{pmatrix} \cdot & 1 & 1 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & 1 & 1 \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix}. \quad (9)$$

In particular, the partitions in (9) identify five sets of edges:

1. A_S , edges between ordered vertices (e.g., empty in (9));

Algorithm 2 Linear algebraic pseudocode for pre-order binary tree traversal. The input tree $\mathcal{G} = (V, E)$ is represented by a vector $s \in \mathbb{N}^N$, $N = |V|$ and the root is in position $\rho \in N$. At each iteration $\sigma_\tau \in V$ represents the top of s_P while $\sigma_L, \sigma_R \in V$ its left and right children, respectively. The function len computes the length of a vector while $\text{children} : V \mapsto V \times V$ computes the children of a vertex.

1: **function** $s = \text{LAPREORDERTREE TRAVERSAL}(s, \rho)$

2: $s \rightarrow \begin{pmatrix} s_S \\ s_P \\ s_U \end{pmatrix}$, where $\text{len}(s_P) = \text{len}(s_S) = 0$

Step 1) Push root to s_P :

3: $\begin{pmatrix} s_S \\ s_P \\ s_U \end{pmatrix} \rightarrow \begin{pmatrix} s_1 \\ s_2 \\ s_3 \\ \sigma_\rho \\ s_4 \end{pmatrix}$

4: $\begin{pmatrix} p_1 \\ p_2 \end{pmatrix} = P^T \begin{pmatrix} s_2 \\ \sigma_\rho \\ s_3 \\ s_4 \end{pmatrix}$, s.t. $p_1 = (\sigma_\rho \ s_2)^T$,
 $p_2 = (s_3 \ s_4)^T$

5: $\begin{pmatrix} s_S \\ s_P \\ s_U \end{pmatrix} \leftarrow \begin{pmatrix} s_1 \\ p_1 \\ p_2 \end{pmatrix}$

6: **while** $\text{len}(s_S) < N$ **do**

7: $\begin{pmatrix} s_S \\ s_P \\ s_U \end{pmatrix} \rightarrow \begin{pmatrix} s_1 \\ \sigma_\tau \\ s_2 \\ s_3 \\ \sigma_L \\ s_4 \\ \sigma_R \\ s_5 \end{pmatrix}$, s.t. $\sigma_L, \sigma_R \leftarrow \text{children}(\sigma_\tau)$

Step 2) Pop:

8: $p_0 = \begin{pmatrix} s_1 \\ \sigma_\tau \end{pmatrix}$

Step 3) Push:

9: $\begin{pmatrix} p_1 \\ p_2 \end{pmatrix} = P^T \begin{pmatrix} s_2 \\ s_3 \\ \sigma_L \\ s_4 \\ \sigma_R \\ s_5 \end{pmatrix}$, s.t. $p_1 = (\sigma_L \ \sigma_R \ s_2)^T$,
 $p_2 = (s_3 \ s_4 \ s_5)^T$

10: $\begin{pmatrix} s_S \\ s_P \\ s_U \end{pmatrix} \leftarrow \begin{pmatrix} p_0 \\ p_1 \\ p_2 \end{pmatrix}$

11: **end while**

12: **end function**

2. A_{SP} , edges connecting ordered vertices to partially ordered ones (e.g., in (9) the two edges connecting 4 with 3 and 5);
3. A_{SU} , edges connecting ordered and unordered vertices (e.g., empty in (9));
4. A_{PU} , edges connecting partially ordered vertices to unordered ones (e.g., in (9) the two edges connecting 5 with 1 and 2);
5. A_U , edges between unordered vertices (e.g., also empty in (9)).

This allows us to redefine Algorithm 2 in terms of permutations of the adjacency matrix as shown in Algorithm 3. In Algorithm 3, Steps 2 and 3 act as follow on the permuted adjacency matrix:

- i) Pop. If $\tau \neq \rho$, this operation moves one edge from A_{SP} to A_S . For example, popping from s in (8), i.e.,

$$\begin{pmatrix} 4 \\ 3 \\ 5 \\ 1 \\ 2 \end{pmatrix} \Rightarrow \begin{pmatrix} 4 \\ 3 \\ 5 \\ 1 \\ 2 \end{pmatrix}$$

is equivalent to the following operation on $A_{\mathcal{G}}$:

$$\begin{pmatrix} \cdot & 1 & 1 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & 1 & 1 \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix} \Rightarrow \begin{pmatrix} \cdot & 1 & 1 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & 1 & 1 \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix},$$

where the edge between 4 and 3 is added to A_S . If σ_τ is not a leaf, the operation also moves one or more edges from A_{PU} to A_{SU} .

- ii) Push. If σ_τ is not a leaf, this operation moves one or more edges from A_{SU} to A_{SP} . If the pushed vertices are not leaves, one or more edges are moved from A_U to A_{PU} . For instance, pushing 1 and 2 onto s_P , i.e.,

$$\begin{pmatrix} 4 \\ 3 \\ 5 \\ 1 \\ 2 \end{pmatrix} \Rightarrow \begin{pmatrix} 4 \\ 3 \\ 5 \\ 1 \\ 2 \end{pmatrix}$$

is equivalent to the following operation on $A_{\mathcal{G}}$:

$$\begin{pmatrix} \cdot & 1 & 1 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & 1 & 1 \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix} \Rightarrow \begin{pmatrix} \cdot & 1 & 1 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & 1 & 1 \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix},$$

where the edges connecting 5 with 1 and 2 are moved from A_{SU} to A_{SP} as vertex 5 is currently popped and 1 and 2 are pushed on s_P .

Algorithm 3 Linear algebraic pre-order tree traversal. The inputs are the vector of vertices s , the adjacency matrix A_G of the tree, and the root $\rho \in V$. The output is the permuted vector s such that σ_i is the i th vertex in the pre-order traversal sequence. The function $\text{BUILD_PERMUTE}(s, n, \tau, \beta)$ creates a permutation matrix P to account for the push of the vertices identified by n given the current state of s provided by the lines $[\tau, \beta)$ and returns the number of children ν .

```

1: function  $s = \text{LAPREORDERTREEADJMAT}(s, A_G, \rho)$ 
2:    $\tau \leftarrow \beta - 1$ 
3:    $P \leftarrow I_N$ 

```

Step 1) Push the root:

```

4:    $\nu, P \leftarrow \text{BUILD\_PERMUTE}(s, e_\rho, \tau, \beta)$ 
5:    $A_G \leftarrow P^T A_G P$ 
6:    $s \leftarrow P^T s$ 
7:    $\beta \leftarrow \beta + \nu$ 

```

```

8:   while  $\tau < N$  do

```

Adjacency row of top vertex:

```

9:      $n \leftarrow A_G^T e_\tau$ 

```

Step 2) Pop:

```

10:     $\tau \leftarrow \tau + 1$ 
11:     $\beta \leftarrow \beta + (\beta < \tau)$  ▷ Keep  $\beta \geq \tau$ 

```

Build permutation matrix:

```

12:     $\nu, P \leftarrow \text{BUILD\_PERMUTE}(s, n, \tau, \beta)$ 

```

Step 3) Push:

```

13:     $A_G \leftarrow P^T A_G P$ 
14:     $s \leftarrow P^T s$ 
15:     $\beta \leftarrow \beta + \nu$ 

```

```

16:  end while

```

```

17: end function

```

At this point, we are left with discussing how Algorithm 3 builds the permutation matrix P .

Building the permutation matrix. We will provide a definition of the permutation that need take place in BUILD_PERMUTE as part of Step 3 by providing the pre- and post-condition for the Hoare triple [3] $\{Pre\}$ Step 3 $\{Post\}$. The precondition for Step 3 requires s to be partitioned and have up

to ν vertices in s_U to be pushed to s_P :

$$Pre \equiv s = \begin{pmatrix} \hat{s}_S \\ \hat{s}_P \\ \hat{s}_U \end{pmatrix}, \quad \nu = n^T n \geq 0, \quad \text{find}(n) \geq \beta, \quad (10)$$

where we denote with $\hat{s}_x$ the state of subvector s_x before the execution of Step 3. The permutation matrix P should be defined such that if Pre holds and Step 3 is executed the following postcondition can be ensured:

$$Post \equiv \hat{s}_S = s_S, \quad (11)$$

$$P^T \begin{pmatrix} 0 \\ \hat{s}_P \\ 0 \end{pmatrix} = s \odot 1_{\tau+\nu \leq j < \beta} = \begin{pmatrix} 0 \\ 0 \\ \hat{s}_P \end{pmatrix}, \quad (12)$$

$$P^T \begin{pmatrix} 0 \\ 0 \\ \hat{s}_U \end{pmatrix} = s \odot (1_{\tau \leq j < \tau+\nu} + 1_{j \geq \beta} - n), \quad (13)$$

$$P^T n = 1_{\tau \leq j < \tau+\nu},$$

which means that the P should not alter the state of s_S (i.e., (11)), should maintain $\hat{s}_P$ as the bottom of s_P (i.e., (12)), and should permute the ν vertices from $\hat{s}_U$ to the top part of s_P (i.e., (13)). One could therefore implement the BUILD_PERMUTE function for the pre-order binary tree traversal as shown in Algorithm 4. The latter, first determines the number ν of vertices to be pushed and the vector ℓ of their locations from the adjacency row n and then computes P . In

Algorithm 4 Pseudocode for the function BUILD_PERMUTE in Algorithm 3. The inputs are the vector of vertices s , an adjacency row n , and the positions τ and β currently delimiting s_P . The function BUILD_PERMUTE computes P to account for pushing the vertices pointed to by edges in n onto the partially ordered region and returns their number ν .

```

function  $\nu, P = \text{BUILD\_PERMUTE}(s, n, \tau, \beta)$ 
   $\nu \leftarrow n^T n$  ▷ Compute number of children
   $\ell \leftarrow \text{find}(n)$  ▷ Compute children's locations
   $P \leftarrow \text{Create } P \text{ s.t. } \{(10)\} \text{ Step 3 } \{(11)-(13)\}$ 
end function

```

general, given the scope of the paper to establish a mathematical approach for DFS computations, it suffices to provide the Hoare assertions to define P , as picking a particular permutation would be more of an implementation-specific task. However, notice that a formulation for P that satisfies the assertions above can be constructed following the steps provided in Sec. 3 for moving ν vertices from the unordered vector s_U to s_P .

4.2 Post-Order Tree Traversal

In traditional stack-based, iterative implementations of DFS for binary trees, different orderings are the consequence of different sequences of push and pop operations. For instance the post-order DFS variant of Algorithm 3 is shown in Algorithm 5. Differently from the pre-order case, the post-order

Algorithm 5 Pseudocode for the iterative, post-order DFS traversal of tree $\mathcal{G} = (V, E)$ with root $\rho \in V$.

```

procedure  $\ell$ : LIST = POSTORDERTREE TRAVERSAL( $\mathcal{G}, \rho$ )
  Stack S
  S.push( $\rho$ )
  while not S.empty() do
     $v \leftarrow$  S.peek()
    if  $\nexists w : (v, w) \in \mathcal{G}.E$  and  $w \notin \ell$  then
       $\ell.append(v)$ 
      S.pop()
    else
      for all  $e \in \mathcal{G}.E : e = (v, w)$  do
        S.push( $w$ )
      end for
    end if
  end while
end procedure

```

traversal visits a parent vertex only after all its children have been visited. To this end, Algorithm 5 performs a pop only after having ascertained that the top of the stack is either a leaf or all its children have been visited. For instance, the post-order traversal sequence of the tree in Fig. 1 is (3 1 2 5 4).

Post-order linear algebraic formulation. The linear algebraic formulation of Algorithm 5 is similar to its pre-order counterpart in Algorithm 3 except that the order of Step 2 and 3 is inverted as follows:

Build permutation matrix:

$$v \leftarrow \text{BUILD_PERMUTE}(s, n, \tau, \beta)$$

Step 2) Push:

$$\begin{aligned} A_{\mathcal{G}} &\leftarrow P^T A_{\mathcal{G}} P \\ s &\leftarrow P^T s \\ \beta &\leftarrow \beta + v \end{aligned}$$

Step 3) Pop:

$$\begin{aligned} \tau &\leftarrow \tau + (v = 0) \\ \beta &\leftarrow \beta + (\beta < \tau) \end{aligned} \quad \triangleright \text{Keep } \beta \geq \tau$$

This is equivalent to moving the top vertex from s_P to s_S only when $v = 0$, i.e., it is a leaf or all of its children are already in s_S .

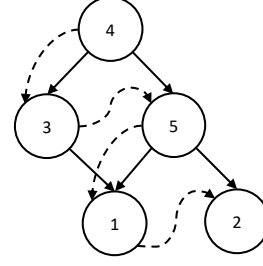


Figure 2. A graph annotated with its preorder traversal (dashed arrows) starting from vertex 4.

Building the permutation matrix. To make sure that $v = 0$ when all of a vertex's children are in s_S , the BUILD_PERMUTE function needs only consider children with position $j \geq \beta$ (i.e., consider only potential edges between the top s_P vertex and unordered vertices). Step 2 is therefore characterized by a Hoare triple in all similar to the one for the pre-order tree traversal with the exception of the precondition that must exclude children in s_S :

$$Pre \equiv s = \begin{pmatrix} s_S \\ s_P \\ s_U \end{pmatrix}, \quad n_U = n \odot 1_{j \geq \beta}, \quad (14)$$

$$v = n_U^T n_U \geq 0, \quad \text{find}(n_U) \geq \beta. \quad (15)$$

And a modified version of Algorithm 4 that follows the assertions for Step 2 of the post-order tree traversal is shown below.

```

function  $v, P = \text{BUILD\_PERMUTE}(s, n, \tau, \beta)$ 
   $n_U \leftarrow n \odot 1_{j \geq \beta}$   $\triangleright$  Only children in  $s_U$ 
   $v \leftarrow n_U^T n_U$   $\triangleright$  Compute number of children
   $\ell \leftarrow \text{find}(n_U)$   $\triangleright$  Compute children's locations
   $P \leftarrow \text{Create } P \text{ s.t. } \{(14)-(15)\}$  Step 2  $\{(11)-(13)\}$ 
end function

```

5 Linear Algebraic DFS on Graphs

When working with trees, we can assume that each vertex has only one parent in a depth-first traversal. However, with more general graphs this assumption no longer holds and each vertex can have multiple predecessors as illustrated in Fig. 2. As an example we take the standard formulation for the DFS-based topological sort of a directed acyclic graph (DAG) $\mathcal{G}$ as presented in Algorithm 6. At each iteration the algorithm pops a vertex v from the top of the stack and checks whether or not it has already been visited. If not, all vertices in its out neighborhood are pushed onto the stack and v is marked as visited. The additional check for whether a given vertex has already been processed before pushing it onto the stack ensures convergence.

Notice that Algorithm 6 allows duplication of values on the stack as vertices with multiple parents may be pushed

Algorithm 6 Pseudocode for the iterative topological sort of a graph $\mathcal{G} = (V, E)$ from root $\rho \in V$.

```

procedure  $\ell$ =PREORDERGRAPHTRAVERSAL( $\mathcal{G}, \rho$ )
  Stack S
  S.push( $\rho$ )
  while not S.empty() do
     $v \leftarrow$  S.pop()
    if  $v \notin \ell$  then
       $\ell.append(v)$ 
      for all  $e \in \mathcal{G}.E : e = (v, w)$  do
        S.push( $w$ )
      end for
    end if
  end while
end procedure

```

multiple times. For instance, vertex 5 in Fig. 2 has two predecessors (i.e., vertices 2 and 3) and would be pushed twice onto the stack. However, the if condition ensures that duplicated values in the stack do not get reprocessed, as they will be marked visited on the first run through.

5.1 Linear Algebraic Topological Sort

Generalizing the linear algebraic approach proposed in Sec. 4 from trees to graphs requires to take into account two major differences that appear from our graph traversal example above.

Generalized partitioning of $A_{\mathcal{G}}$. No *a priori* assumptions can be made regarding how edges connect vertices of different statuses. Recall that the linear algebraic formulation for binary trees in Algorithm 3 is based on the assumption that vertices in s_P and s_U can only connect with unordered vertices, i.e., the four bottom left submatrices in (9) are zero matrices. For instance, consider the following intermediate state of traversing the the graph in Fig. 2 using Algorithm 6,

$$s = \left(\begin{array}{c} \overline{s_S} \\ \overline{s_P} \\ \overline{s_U} \end{array} \right) = \left(\begin{array}{c} 1 \\ 2 \\ 4 \\ 5 \\ 3 \end{array} \right). \quad (16)$$

The state of s above reflects the scenario where vertices 1 and 2 have been processed and vertices 4 and 5 have been brought into s_P right after popping 2 out of it. If we look at the permuted adjacency matrix A associated to s we would recognize that an edge is now available between vertices partially ordered (i.e., 3 to 5) as well as between a partially

ordered vertex and an ordered one (i.e., 4 to 1):

$$A = P^T A_{\mathcal{G}} P = \begin{array}{c} \begin{array}{ccccc} & 1 & 2 & 4 & 5 & 3 \\ \begin{array}{c} 1 \\ 2 \\ 4 \\ 5 \\ 3 \end{array} & \left(\begin{array}{cc|cc|c} \cdot & 1 & \cdot & \cdot & 1 \\ \cdot & \cdot & 1 & 1 & \cdot \\ \hline 1 & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & 1 & \cdot \end{array} \right) \end{array}.$$

In general, no partition of A can be considered empty for a general graph, changing the partitioning scheme of A from (9) to the following one,

$$A = \left(\begin{array}{c|c|c} A_S & A_{SP} & A_{SU} \\ \hline A_{PS} & A_P & A_{PU} \\ \hline A_{US} & A_{UP} & A_U \end{array} \right). \quad (17)$$

Handling multiple predecessors. Vertices in a graph could have multiple predecessors. A stack-based approach such as in Algorithm 6 will allow a vertex v with multiple predecessors to be inserted multiple times into the stack. As such, an additional check is required to ensure that v is visited only once. With this check in place, only the latest push of v onto the stack will be visited. Subsequent encounters of v from the stack will be ignored.

We make the following two observations: 1) the stack semantics require multiple pushes of v onto the stack, and the insertion of the check. If one could “dive” into the stack to remove previous instances of v , the check would be unnecessary as there will always be at most one instance of v on the stack, and 2) once v has been popped from the stack, it cannot be reinserted onto the stack since vertices can only be visited once.

The observations allow us to define a linear algebraic approach for topological sort based on the same scheme provided in Algorithm 3 where the BUILD_PERMUTE function is constructed with the following conditions:

- (Cond a) Vertices in the ordered region s_S cannot be placed back into the partially ordered region s_P . This results from the fact that a vertex that has been visited cannot be visited again.
- (Cond b) Vertices that are already in the partially ordered region s_P are moved to the beginning of s_P while maintaining all other vertices in s_P in their partial order. This is equivalent to removing the previous instance of v , and then pushing a new instance of v onto the top of the stack.
- (Cond c) Vertices that are in s_U are inserted at the top of s_P in the usual fashion.

Algorithm 7 provides a scheme for the BUILD_PERMUTE function based on the above requirements.

6 Biconnected Components of Graphs

The biconnected components of a graph are subgraphs that remain connected if any vertex is removed from them. For

Algorithm 7 Pseudocode for the function BUILD_PERMUTE in the context of topological sort. Inputs are the vector of vertices s to be updated to account for pushing the vertices identified by the adjacency vector n onto the partially ordered region. The function returns the permutation matrix P and the number v_U of newly pushed vertices.

function $v_U, P = \text{BUILD_PERMUTE}(s, n, \tau, \beta)$
 $n_{PU} \leftarrow n \odot 1_{j>\tau}$ $\triangleright$ mask away visited neighbors
 $n_U \leftarrow n \odot 1_{j>\beta}$ $\triangleright$ new vertices to be inserted in s_P
 $v_U \leftarrow n_U^T n_U$ $\triangleright$ number of new vertices in s_P
 $\ell \leftarrow \text{find}(n_{PU})$ $\triangleright$ Compute neighbors' locations
 $P \leftarrow \text{Create } P \text{ based on Sec. 5.1, Cond (a)–(c)}$
end function

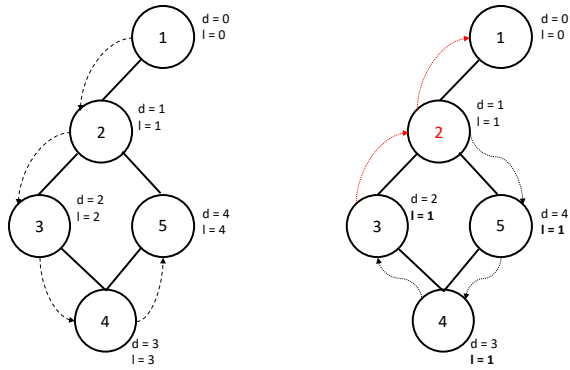


Figure 3. Left: A graph annotated with the depth and low-point based on the depth-first traversal starting from $\rho = 1$ denoted with dashed arrows. Right: The low-point of each vertex is updated as described in Steps 2 and 3. The red arrows identify the situations when the condition in Alg. 8, line 16 holds and `isArticulation` is set to true. Vertex 2 is identified as the only cut vertex due to the condition in Alg. 8, lines 23–24.

example, the graph in Fig. 3, Right contains two biconnected components, i.e., the subgraph containing vertices $\{1, 2\}$ and the subgraph containing vertices $\{2, 3, 4, 5\}$. Vertices that join biconnected components in a connected graph are called cut vertices (e.g., vertex 2 in Fig. 3, Right). The Hopcroft-Tarjan algorithm [4] uses a depth-first traversal to identify cut vertices of a graph.

6.1 Recursive Algorithm to Identify Cut Vertices

In this section, we refer to children and descendants of vertex with respect to the tree obtained traversing the graph depth-first. As such, the children of a vertex could be a subset of its neighbors. The pseudocode for computing the cut vertices of an undirected graph $\mathcal{G}$ is given in Algorithm 8. The algorithm computes the list of cut vertices for $\mathcal{G}$ from a random vertex v of the graph for which the depth is initially set to $d = 0$ (i.e., $c = \text{CUT_VERTICES}(\mathcal{G}, v, 0)$). It progresses recursively

through the graph annotating each vertex with two values: depth and low-point. The low-point is the lowest depth of all descendants of a vertex, i.e., the vertices reachable from a given vertex including itself. There are two cases when a vertex qualifies as a cut vertex:

1. The vertex v is a root with at least two children;
2. The vertex v is not the root and has a descendant w such that $\text{low-point}(w) \geq \text{depth}(v)$.

These vertices are identified when the DFS traversal returns to them after having processed all their children. Algorithm 8 can be broken down into the following key steps:

- Step 1)** Vertex v initializes itself, and makes it known that it is the parent of its children. The algorithm then recursively calls `CUTVERTICES` on each of its children;
- Step 2)** After the recursive call returns, check whether vertex v has any children with a low-point greater than its depth. Update the low-point of vertex v with the minimum of its current low-point and that of all its children;
- Step 3)** For neighbors that are neither children nor the parent of vertex v , update the low-point of vertex v with the minimum of its current low-point and the depth of of all such neighbors;
- Step 4)** Check if vertex v is a cut vertex (see definition above) and if so, update the list of cut vertices.

Fig 3 shows an example for how the algorithm computes the cut vertices of a graph. Vertex 2 is identified as the only cut vertex. Although `isArticulation` is set to true for vertex 1, it is not marked as a cut vertex as it is a root with a single child.

6.2 Linear Algebraic Formulation

We focus on the DFS aspect of the Hopcraft-Tarjan algorithm. Notice that in Algorithm 8 each neighbor of a vertex v is processed individually (line 11). Dependent on whether the neighbor has been visited (i.e., it is a child of vertex v), different actions are taken. Specifically, if the neighbor vertex has not been visited, the function is recursively called on the neighboring vertex. When all neighboring vertices have been processed, the algorithm then processes the current vertex v and determines if it is a cut vertex. Once this is determined, the vertex v is no longer processed by the algorithm.

These observations inform us of the following:

1. Vertices that are pushed onto the partially ordered region have not been visited. This implies that these vertices must be from the unordered region of our adjacency matrix. This implies that

$$n_U \leftarrow n \odot 1_{j \geq \beta}, \quad (\text{Algorithm 10, Line 13}).$$

2. Neighbors are processed individually. This means that at most one unsorted vertex is moved into the partially ordered region at a time. This implies that $v \leq 1$ in Algorithm 10, Line 22.

Algorithm 8 Pseudocode for the recursive computation of cut vertices in a connected undirected graph $\mathcal{G} = (V, E)$.

```

1: for all  $v \in \mathcal{G}.V$  do                                ▶ Initial state
2:    $\text{visited}[v] \leftarrow \text{false}$ 
3:    $\text{depth}[v] \leftarrow \text{low}[v] \leftarrow 0$ 
4:    $\text{parent}[v] \leftarrow \text{null}$ 
5: end for

6: function  $\text{C} : \text{LIST} = \text{CUTVERTICES}(\mathcal{G}, v, d)$ 
7:    $\text{visited}[v] \leftarrow \text{true}$ 
8:    $\text{depth}[v] \leftarrow \text{low}[v] \leftarrow d$ 
9:    $\text{childCount} \leftarrow 0$ 
10:   $\text{isArticulation} \leftarrow \text{false}$ 
11:  for all  $w \in \mathcal{G}.V : (v, w) \in \mathcal{G}.E$  do
12:    if not  $\text{visited}[w]$  then
13:       $\text{parent}[w] \leftarrow v$ 
14:       $\text{CUTVERTICES}(\mathcal{G}, w, d + 1)$ 
15:       $\text{childCount} \leftarrow \text{childCount} + 1$ 
16:      if  $\text{low}[w] \geq \text{depth}[v]$  then
17:         $\text{isArticulation} \leftarrow \text{true}$ 
18:      end if
19:       $\text{low}[v] \leftarrow \min(\text{low}[v], \text{low}[w])$ 
20:    else if  $w \neq \text{parent}[v]$  then
21:       $\text{low}[v] \leftarrow \min(\text{low}[v], \text{depth}[w])$ 
22:    end if
23:    if  $(\text{parent}[v] \neq \text{null} \text{ and } \text{isArticulation})$  or
24:       $(\text{parent}[v] = \text{null} \text{ and } \text{childCount} > 1)$  then
25:       $c.\text{append}(v)$ 
26:    end if
27:  end for
28: end function

```

Algorithm 9 Pseudocode for the function BUILD_PERMUTE in the context of computing the cut vertices. Inputs are the vector of vertices s the position of a vertex to be placed on top of the partially ordered region s_P . The function returns the permutation matrix and the number ν of newly pushed vertices (in this case zero or one).

```

function  $\nu, P = \text{BUILD\_PERMUTE}(s, \lambda, \tau, \beta)$ 
   $\nu \leftarrow (\lambda > 0)$ 
   $P \leftarrow \text{Create } P \text{ based on Sec. 5.1, Cond. (a)–(c)}$ 
   $\text{and only considering the child in position } \lambda$ 
end function

```

- When all neighboring vertices of vertex v have been processed, vertex v no longer participates in the algorithm. It is at this time that vertex v is moved to the ordered region and removed from the partially ordered one. In Algorithm 10, this is equivalent to

$$\tau = \tau + 1, \quad (\text{Algorithm 10, Line 40}).$$

All other linear algebra operations described in Alg. 10 are related to the processing of the different vertices and the

booking of information such as the parent vertices (matrix M), the low-point vector l , the depth vector d , and the vector of cut-vertices c .

Algorithm 10 Linear Algebraic formulation of the Hopcroft-Tarjan algorithm. The inputs are the vector of vertices s , the adjacency matrix of the graph $\mathcal{G}$, and the starting vertex ρ . The output is a vector c of size N with the cut vertices at positions $\text{find}(c)$.

```

1: function  $c = \text{LACUTVERTICES}(s, \mathcal{G}, \rho)$ 
2:    $\tau \leftarrow \beta \leftarrow 1$ 
3:    $P \leftarrow I_N$ 
4:    $M \leftarrow 0$ 
5:    $c \leftarrow d \leftarrow l \leftarrow 0$ 
6:    $\nu, P \leftarrow \text{BUILD\_PERMUTE}(s, \rho, \tau, \beta)$                                 ▶ See Alg. 9
7:    $\beta \leftarrow \beta + \nu$ 
8:   while  $\tau < N$  do
9:      $A_{\mathcal{G}} \leftarrow P^T A_{\mathcal{G}} P$ 
10:     $(s \ l \ d) \leftarrow P^T \cdot (s \ l \ d)$ 
11:     $M \leftarrow P^T M P$ 
12:     $n \leftarrow B^T e_{\tau}$ 
13:     $n_U \leftarrow n \odot 1_{j \geq \beta}$ 
14:    if  $n_U \neq 0$  then
15:       $\ell_U \leftarrow \text{find}(n_U)$ 
16:       $\lambda \leftarrow \min(\ell_U)$ 
17:       $d \leftarrow d + e_{\lambda} \cdot (d^T e_{\tau} + 1)$ 
18:       $l \leftarrow l + e_{\lambda} \cdot (d^T e_{\tau} + 1)$ 
19:       $M \leftarrow M + e_{\tau} \cdot e_{\lambda}^T$ 
20:    else  $\lambda \leftarrow 0$ 
21:    end if
22:     $\nu, P \leftarrow \text{BUILD\_PERMUTE}(s, \lambda, \tau, \beta)$ 
23:     $\beta \leftarrow \beta + \nu$ 
24:    if  $\nu = 0$  then
25:       $n_c \leftarrow M^T e_{\tau}$ 
26:       $\nu_c \leftarrow n_c^T n_c$ 
27:       $\lambda_{\tau} \leftarrow l^T e_{\tau}$ 
28:       $\text{min\_low} \leftarrow \min(l \odot n_c + \lambda_{\tau}(1_N - n_c))$ 
29:       $\text{max\_low} \leftarrow \max(l \odot n_c)$ 
30:       $\alpha \leftarrow (\text{max\_low} \geq d^T e_{\tau})$ 
31:       $l \leftarrow l \odot (1_N - e_{\tau}) + \text{min\_low} \cdot e_{\tau}$ 
32:       $e_p \leftarrow M e_{\tau}$ 
33:       $n_{SP} \leftarrow (n \odot 1_{1 \leq j < \beta}) - e_p$                                 ▶ exclude parent
34:       $\text{min\_depth} \leftarrow \min(d \odot n_{SP} + \lambda_{\tau}(1_N - n_c))$ 
35:       $l \leftarrow l \odot (1_N - e_{\tau}) + \text{min\_depth} \cdot e_{\tau}$ 
36:      if  $(e_p \neq 0 \text{ and } \alpha \neq 0)$  or
37:         $(e_p = 0 \text{ and } \nu_c > 1)$  then
38:         $c \leftarrow c + s \odot e_{\tau}$ 
39:      end if
40:       $\tau \leftarrow \tau + 1$ 
41:       $\beta \leftarrow \beta + (\beta < \tau)$ 
42:    end if
43:  end while
44: end function

```

7 Conclusion

In this paper, we presented a number of graph algorithms, formulated in the language of linear algebra, that utilizes the depth-first-search (DFS) traversal pattern. Using matrix and vector permutations in the form of pre- and post-multiplying the matrices and vectors with a permutation matrix, we demonstrate an approach to formulating DFS-based graph algorithms in the language of linear algebra. This addresses a major concern with the expressiveness of graph computations in terms of linear algebra building blocks. We believe that this mechanism should be further studied as a possible primitive within current efforts to provide standard interfaces for the implementation of linear algebraic graph programs, such as the GraphBLAS. Future work, include the study of the practical impact of our approach in the context of real-world graph applications.

Acknowledgement

This work was sponsored partly by the DARPA BRASS program under agreement FA8750-16-2-003, and by the NSF through award ACI 1550486. The content, views and conclusions presented in this document are those of the authors and do not necessarily reflect the position or the policy of the sponsoring agencies.

References

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. *Introduction to Algorithms, Third Edition* (3rd ed.). The MIT Press.
- [2] Timothy Davis. 2018. Algorithm 9xx: SuiteSparse: GraphBLAS: graph algorithms in the language of sparse linear algebra. (2018).
- [3] Charles Antony Richard Hoare. 1969. An axiomatic basis for computer programming. *Commun. ACM* 12, 10 (1969), 576–580.
- [4] John Hopcroft and Robert Tarjan. 1973. Algorithm 447: efficient algorithms for graph manipulation. *Commun. ACM* 16, 6 (1973), 372–378.
- [5] Jeremy Kepner, Peter Aaltonen, David Bader, Aydın Buluç, Franz Franchetti, John Gilbert, Dylan Hutchison, Manoj Kumar, Andrew Lumsdaine, Henning Meyerhenke, et al. 2016. Mathematical foundations of the GraphBLAS. *arXiv preprint arXiv:1606.05790* (2016).
- [6] Jeremy Kepner and John Gilbert. 2011. *Graph algorithms in the language of linear algebra*. SIAM.
- [7] David J King and John Launchbury. 1995. Structuring depth-first search algorithms in Haskell. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. ACM, 344–354.
- [8] Tze Meng Low, Varun Nagaraj Rao, Matthew Lee, Doru Popovici, Franz Franchetti, and Scott McMillan. 2017. First look: Linear algebra-based triangle counting without matrix multiplication. In *2017 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–6.
- [9] Tze Meng Low, Daniele G. Spampinato, Anurag Kutuluru, Upasana Sridhar, Doru Thom Popovici, Franz Franchetti, and Scott McMillan. 2018. Linear Algebraic Formulation of Edge-centric K-truss Algorithms with Adjacency Matrices. 1–7. <https://doi.org/10.1109/HPEC.2018.8547718>
- [10] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D Owens. 2016. Gunrock: A high-performance graph processing library on the GPU. In *ACM SIGPLAN Notices*, Vol. 51. ACM, 11.
- [11] Marcin Zalewski, Peter Zhang, Andrew Lumsdaine, and Scott McMillan. [n. d.]. cmu-sei/gbtl. <https://github.com/cmu-sei/gbtl>